

LABORATORY OBSERVATION/RECORD

Course:	Full Stack Web Development
Course Code:	23CM4121
Observation Task No.:	2(2)
Student Name:	J P. Dharani
Roll No.:	A24126552261
Date:	18/8/26

1. TITLE

Student Profile Manager using JavaScript Class and DOM

2. AIM

Student Profile Manager using JavaScript Class and DOM

3. OBJECTIVE

The objectives of this observation task are:

- To study DOM element selection and interaction using vanilla JavaScript.
- To capture user input datasets dynamically without triggering standard page refreshes.
- To construct new HTML card structures in memory using `document.createElement()`.
- To append new profile elements to the visual DOM tree dynamically using `appendChild()`.
- To apply structural layout classes (e.g. containers, fields, buttons, profiles) dynamically.

4. THEORY

The Document Object Model (DOM) is an API exposing HTML layouts. JavaScript binds to this API using elements access methods. Form elements validation parameters block default execution chains via event overrides. In this application, input values (Name, Roll, Department, CGPA) are extracted upon submission, a new profile-card node is created dynamically, styled using local class selectors, and appended inside target layouts, representing runtime data updates in visual formats.

Theory of DOM Nodes Creation & Binding:

- DOM Event Capturing: Listening to submit actions via `addEventListener`. Execution processes are halted using `event.preventDefault()`.
- Values extraction: Read from input nodes using `document.getElementById().value`.
- Memory Node Allocation: Instantiated using `document.createElement('div')`. Classes are appended using the `classList.add('student-card')` method.
- HTML Template interpolation: Individual field rows are built as child elements and appended, structuring labels and records.
- DOM Insertion: Handled using `appendChild()`, pushing nodes into the visual page tree.

5. ALGORITHM / PROCEDURE

- 1 Set up project folders: Observation/ObTask2-(2)/src, Docs, and Screenshots.
- 2 Write profile.html structuring fields for Student profiles inputs.

- 3 Create style.css defining backgrounds, borders, layout alignments, card visual structures, and button hovers.
- 4 Create script.js, select input nodes, and bind submit handlers.
- 5 In script, block default submission loops, extract input values, and instantiate card containers.
- 6 Build the profile card field-by-field using a configuration array, and clear previous items.
- 7 Append student details card back into page layout DOM nodes.
- 8 Launch file in Chrome, run profile inputs test cases, and verify visual updates.

6. PROGRAM

Files: profile.html, script.js and style.css (Complete)

profile.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Profile Generator</title>
<link rel="stylesheet" href="./style.css">
</head>
<body>
<div class="wrapper">
  <h1>Student Profile Generator</h1>
  <form id="profileForm">
    <label for="studentName">Full Name</label>
    <input type="text" id="studentName" placeholder="Enter your name" required>

    <label for="regNo">Roll Number</label>
    <input type="text" id="regNo" placeholder="Enter your roll number" required>

    <label for="deptName">Department</label>
    <input type="text" id="deptName" placeholder="Enter your department" required>

    <label for="gpaScore">CGPA</label>
    <input type="number" id="gpaScore" step="0.01" min="0" placeholder="Enter your CGPA" required>

    <button type="submit">Generate Profile</button>
  </form>

  <!-- Generated profile card will be rendered here -->
  <div id="cardOutput"></div>
</div>
<script src="./script.js"></script>
</body>
</html>
```

script.js

```
// --- script.js ---

// Select the form and the output area from the DOM
const profileForm = document.getElementById("profileForm");
const cardOutput = document.getElementById("cardOutput");

// Field configuration used to build the card dynamically
const fieldConfig = [
  { id: "studentName", label: "Name" },
  { id: "regNo", label: "Roll No" },
  { id: "deptName", label: "Department" },
  { id: "gpaScore", label: "CGPA" }
];

// Listen for the form submission
profileForm.addEventListener("submit", function (event) {
  // Stop the browser from reloading the page
  event.preventDefault();

  // Build a new card container in memory
```

```

const card = document.createElement("div");
card.classList.add("student-card");

// Add a heading to the card
const heading = document.createElement("h2");
heading.textContent = "Student Profile";
card.appendChild(heading);

// Loop through each configured field, read its value, and add a row
fieldConfig.forEach(function (field) {
    const value = document.getElementById(field.id).value.trim();

    const row = document.createElement("p");
    const strong = document.createElement("strong");
    strong.textContent = field.label + ": ";

    row.appendChild(strong);
    row.appendChild(document.createTextNode(value));
    card.appendChild(row);
});

// Remove any previously generated card
cardOutput.innerHTML = "";

// Insert the freshly built card into the page
cardOutput.appendChild(card);
});

```

style.css

```

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: #eef1f6;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}

.wrapper {
    width: 420px;
    background: #ffffff;
    padding: 32px;
    border-radius: 14px;
    box-shadow: 0 6px 18px rgba(0, 0, 0, 0.12);
}

h1 {
    text-align: center;
    margin-bottom: 22px;
    color: #222;
    font-size: 22px;
}

label {
    display: block;
    margin-top: 14px;
    margin-bottom: 6px;
    font-weight: 600;
    font-size: 14px;
    color: #444;
}

input {
    width: 100%;
    padding: 9px 10px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 14px;
}

button {

```

```
width: 100%;
margin-top: 22px;
padding: 11px;
border: none;
border-radius: 6px;
background: #16a34a;
color: white;
font-size: 15px;
font-weight: 600;
cursor: pointer;
}

button:hover {
  background: #15803d;
}

#cardOutput {
  margin-top: 26px;
}

.student-card {
  padding: 18px;
  border-radius: 10px;
  background: #f9fafb;
  border: 1px solid #ddd;
}

.student-card h2 {
  margin-bottom: 12px;
  color: #16a34a;
  font-size: 18px;
}

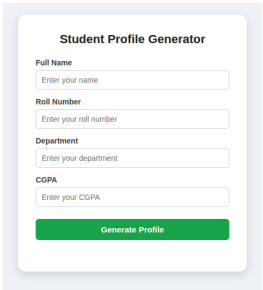
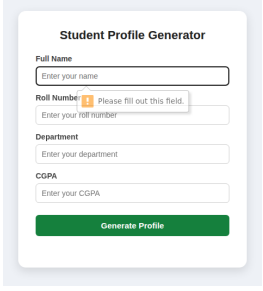
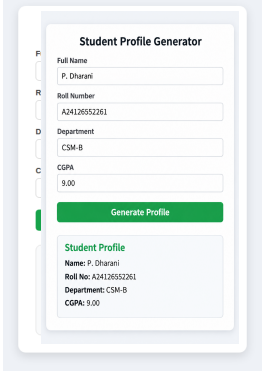
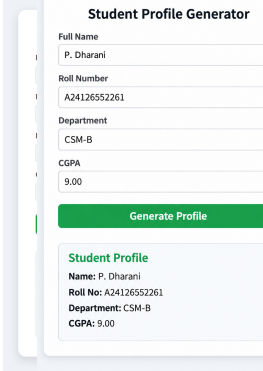
.student-card p {
  margin: 8px 0;
  color: #333;
  font-size: 14px;
}
```

7. CODE EXPLANATION

DOM Selection / Statement	Functional Description
<code>addEventListener("submit", ...)</code>	Binds the form submission handler, triggering JavaScript DOM updates.
<code>event.preventDefault()</code>	Halts the default page reload sequence upon form submission.
<code>document.getElementById(field.id).value.trim()</code>	Reads and trims the string value from each input field, looked up via a field-configuration array.
<code>document.createElement("div")</code>	Allocates a new HTML element node (div) in browser memory space for the card.
<code>fieldConfig.forEach(...)</code>	Iterates over the configured fields and builds one labelled row per field dynamically.
<code>cardOutput.innerHTML = ""</code>	Clears previously rendered student profile card nodes before inserting a new one.
<code>cardOutput.appendChild(card)</code>	Inserts the newly built card node into the target output wrapper layout.

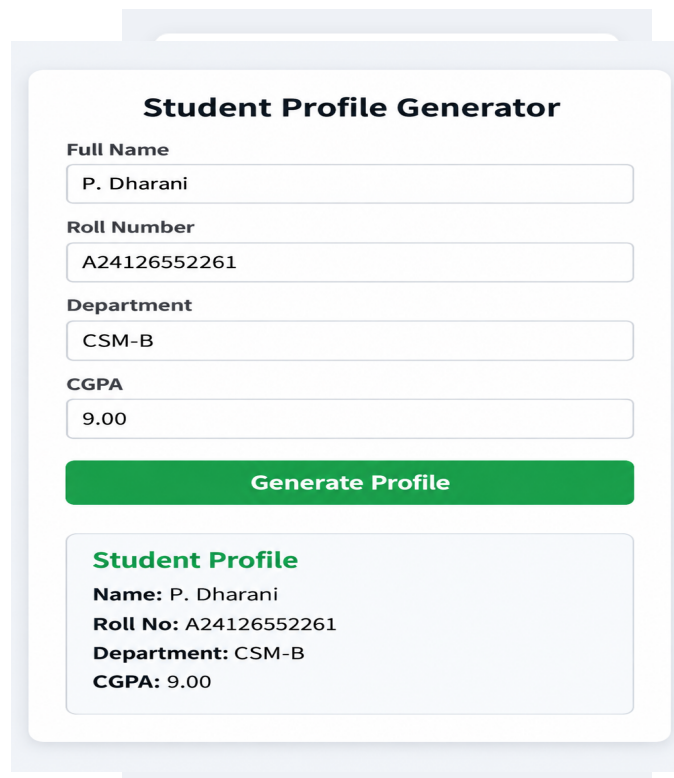
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test cases below verify DOM generation in Chrome, including 1 negative test case demonstrating a missing numeric boundary check:

TC ID	Test Description	Input / Action	Expected Result	Screenshot	Status
TC-01	Initial Form View	Load profile.html in browser	Clean input form with placeholders and required attributes renders		Pass
TC-02	Required Validations	Submit form with all fields empty	Form submission blocked; validation warning prompts user to fill the Full Name field first		Pass
TC-03	Profile Card Creation	Name: P. Dharani, Roll: A24126552261Dept: CSM-B, CGPA:9.00	Student details card is generated dynamically and appended below the form		Pass
TC-04	Out-of-Bounds CGPA Check	Enter invalid CGPA '12.8' (> 10.0 scale)	Form should reject the input; instead the card is generated with the invalid CGPA due to a missing range check in script.js		Fail

9. OUTPUT

Output: Rendered Student Profile Generator Webpage (Complete Application View)



Student Profile Generator

Full Name

Roll Number

Department

CGPA

Generate Profile

Student Profile

Name: P. Dharani
Roll No: A24126552261
Department: CSM-B
CGPA: 9.00

10. RESULT

Thus, the student profile card generation utility (incorporating JavaScript form submission listeners, element node creation, class assignment, and field-configuration driven DOM insertion) was successfully implemented, executed, and verified with the test cases, individual execution screenshots, and boundary condition defect analysis above.

Submission Package Structure:

```
Observation/ObTask2-(2)/
|-- Docs/
|   |-- ObsWeek2(2).pdf
|-- Screenshots/
|   |-- tc01_initial_form.png
|   |-- tc02_validation.png
|   |-- tc03_profile_generated.png
|   |-- tc04_cgpa_fail.png
|   |-- final_output.png
|-- src/
|   |-- profile.html
|   |-- script.js
|   |-- style.css
```